Quantum Inf Process (2018) 17:169
https://doi.org/10.1007/s11128-018-1908-8

Mapping from multiple-control Toffoli circuits to linear nearest neighbor quantum circuits

Xueyun Cheng[1,2] · **Zhijin Guan**[2] · **Weiping Ding**[2]

Received: 9 January 2018 / Accepted: 19 April 2018
© Springer Science+Business Media, LLC, part of Springer Nature 2018

Abstract In recent years, quantum computing research has been attracting more and more attention, but few studies on the limited interaction distance between quantum bits (qubit) are deeply carried out. This paper presents a mapping method for transforming multiple-control Toffoli (MCT) circuits into linear nearest neighbor (LNN) quantum circuits instead of traditional decomposition-based methods. In order to reduce the number of inserted SWAP gates, a novel type of gate with the optimal LNN quantum realization was constructed, namely NNTS gate. The MCT gate with multiple control bits could be better cascaded by the NNTS gates, in which the arrangement of the input lines was LNN arrangement of the MCT gate. Then, the communication overhead measurement model on inserted SWAP gate count from the original arrangement to the new arrangement was put forward, and we selected one of the LNN arrangements with the minimum SWAP gate count. Moreover, the LNN arrangement-based mapping algorithm was given, and it dealt with the MCT gates in turn and mapped each MCT gate into its LNN form by inserting the minimum number of SWAP gates. Finally, some simplification rules were used, which can further reduce the final quantum cost of the LNN quantum circuit. Experiments on some benchmark MCT circuits indicate that the direct mapping algorithm results in fewer additional SWAP gates in about 50%, while the average improvement rate in quantum cost is

✉ Xueyun Cheng
chen.xy@ntu.edu.cn

Zhijin Guan
guan.zj@ntu.edu.cn

Weiping Ding
ding.wp@ntu.edu.cn

[1] College of Electronics and Information, Nantong University, Nantong, Jiangsu, China

[2] College of Computer Science and Technology, Nantong University, Nantong, Jiangsu, China

Published online: 29 May 2018

16.95% compared to the decomposition-based method. In addition, it has been verified that the proposed method has greater superiority for reversible circuits cascaded by MCT gates with more control bits.

Keywords Reversible circuit · Quantum circuit · Multiple-control Toffoli (MCT) gate · Linear nearest neighbor (LNN) · LNN arrangement · Mapping algorithm

1 Introduction

QUANTUM computing can offer significantly higher performance on certain problems compared to classical computing, for example, integer factorization [1] and unstructured database search [2]. The quantum algorithms are implemented with the help of quantum circuits [3]. Because it is hard or error prone to manipulate more than two qubits in most quantum technologies, only 1-qubit and 2-qubit gates are supported [4]. From a logical point of view, any qubit can interact with each other, but in most cases, 2-qubit gates can only be performed between nearest neighbor qubits. This restriction imposes some communication overhead on the total circuit latency [5] that this can also increase error probability [4]. Some technologies such as one-dimensional ion trap only support the linear nearest neighbor (LNN) architecture of circuits in which the control and target bit of a gate must be adjacent. Therefore, the synthesis of LNN circuits is of practical significance.

Many quantum algorithms include a Boolean component. The synthesis is usually performed by two steps. First, the Boolean component is initially converted into a cascade of reversible gates using some well-known synthesis and optimization methods [6, 7]. Next, this circuit is mapped into a quantum gate cascade using some quantum gate library (e.g., NCV [1]). MCT network synthesis can handle larger functions and circuits [8, 9], and some decomposition methods of MCT gates have been presented. In [10], a new heuristic method to decompose an MCT gate of arbitrary size into the elementary quantum gates was discussed, and the approach avoided exploring all possible partitions as used in [11], which indicated different decomposition strategies have a great influence on the final results. Under the nearest neighbor constraint, all the 2-qubit quantum gates interact on adjacent qubits by inserting a sequence of SWAP gates.

Some methods for mapping an arbitrary quantum circuit into its equivalent circuit on the LNN architectures have been presented, and SWAP gates must be applied for each non-adjacent quantum gate [12–15]. Approaches such as [12, 13] often lead to a significant increase in the quantum cost. In [14], an exact synthesis approach considering the nearest neighbor cost (NNC) with minimal quantum cost was proposed, which could get better quantum circuit realization for the LNN architectures. Ref. [15] utilized the interaction graph to partition the quantum circuit into a set of subcircuits, the number of internal qubit reordering and qubit reordering between the consecutive subcircuits was minimized, which can lead to a considerable reduction in the number of SWAP gates.

The optimal LNN quantum circuit realization of the nearest neighbor Toffoli (NNT) gate was given in [14]. The Toffoli gate-based decomposition of an MCT gate with

multiple control bits (more than two) was presented in [16], and the corresponding NNT-based cascade was put forward here. To reduce the number of inserted SWAP gates, a novel type of gate, NNTS gate was introduced in this paper, and the above-mentioned MCT gate with multiple control bits can be well realized by NNTS gates in the LNN architecture; the arrangement of the input lines at this point is the LNN arrangement of the MCT gate. When we map an MCT circuit into an LNN quantum circuit, there is no need to decompose each MCT gate into the elementary quantum gates at first.

Instead, we can make each MCT gate transform into LNN arrangement form by adding SWAP gates before it, and then substitute each LNN arrangement with the optimal LNN quantum realization. During substitution, some simplification rules were put forward to further decrease the quantum cost of the final LNN quantum circuit. The experimental results show the effectiveness of the proposed algorithms, compared with the direct decomposition methods.

The paper is organized as follows. First, the required fundamental knowledge on reversible as well as quantum circuits is provided and LNN arrangement of any MCT gate is given in Sect. 2. Afterward, the communication overhead measurement model is put forward in Sect. 3. The details of the proposed LNN arrangement-based mapping algorithms are described in Sect. 4. Some simplification rules are introduced in Sect. 5. Section 6 shows the experimental results on some benchmark MCT circuits, and Sect. 7 concludes this paper.

2 Basic concepts

A Boolean function $f: B^n \rightarrow B^n$ is said to be reversible if each input assignment maps to a unique output assignment.

A reversible circuit realizes a reversible function and is a cascade of reversible gates without any feedback or fan-out [3].

Several different reversible gates have been introduced including the Toffoli gate [17], the Fredkin gate [18], and the Peres gate [19].

In the following, we focus on the generalized Toffoli gate with arbitrary number of control lines, which is also known as MCT gate, denoted as $T_{m+1}(C; t)$ or $\mathrm{MCT}_{m+1}(C; t)$, which is a type of universal gate that inverts the state of the target line t when the set of control lines C are all one, m indicates the number of control bits, and $m+1$ is the size of the gate. MCT_1, i.e., NOT gate, always inverts the target line. MCT_2 is called a CNOT gate. MCT_3 is the original Toffoli gate with two control lines.

Figure 1 illustrates a benchmark reversible circuit 4gt5_75, consisting of five MCT gates, where • represents a control line and $\oplus$ is used to indicate the target line of an MCT gate.

A quantum bit, qubit, represents a quantum state with two basic states $|0\rangle$ and $|1\rangle$, and a qubit can get any linear combination of its basic states as follows:

$$|\emptyset\rangle = \alpha|0\rangle + \beta|1\rangle, \tag{1}$$

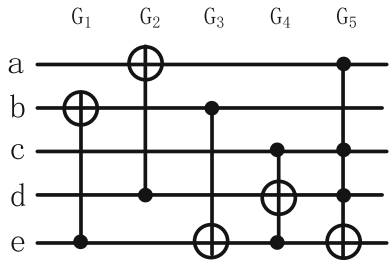


Fig. 1 Optimal MCT circuit of benchmark 4gt5_75

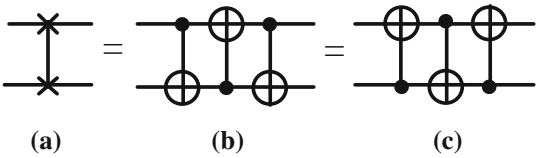


Fig. 2 SWAP gate. **a** Graphical representation, **b** the first circuit realization, and **c** the second circuit realization

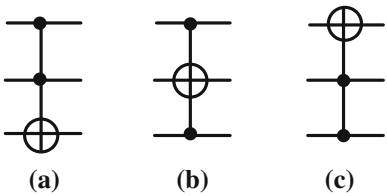


Fig. 3 Three forms of the original Toffoli gate. **a** NNT_3^1 gate, **b** NNT_3^2 gate, and **c** NNT_3^3 gate

where α and β are complex numbers satisfied with

$$|\alpha|^2 + |\beta|^2 = 1. \tag{2}$$

An n-qubit quantum gate performs a specific $2^n \times 2^n$ unitary operation on selected n qubits. Two or more quantum gates can be cascaded to construct a quantum circuit.

Several quantum gates to realize quantum operations have been proposed in the literature, such as NCV gates [16], Clifford+T gates [3]. In this paper, we consider quantum gates from the NCV library which includes NOT gate, CNOT gate, the controlled-V (CV) gate, and the controlled-V^+ (CV^+) gate.

A SWAP gate is a reversible gate which exchanges the value of two qubits, such as i and j, and its symbolic representation is $i \leftrightarrow j$. A SWAP gate can be implemented using three CNOT gates as shown in Fig. 2 [3]. Hence, its quantum cost (QC) is 3.

If a SWAP gate operates on two adjacent qubits, it is an LNN SWAP gates, denoted as NNSwap. Generally, NNSwap gates are inserted before each non-adjacent gate g in order to "move" the control (target) line of g toward the target (control) line until they become adjacent.

For any MCT_3 gate, we can insert NNSwap gates before it to make its control bits and target bit to be placed on adjacent lines, i.e., NNT gate, which has three forms, as shown in Fig. 3a–c, denoted as NNT_3^1, NNT_3^2, and NNT_3^3, respectively.

The best reported LNN quantum realization of NNT_3^1 gate has quantum cost of 9 [14], as shown in Fig. 4.

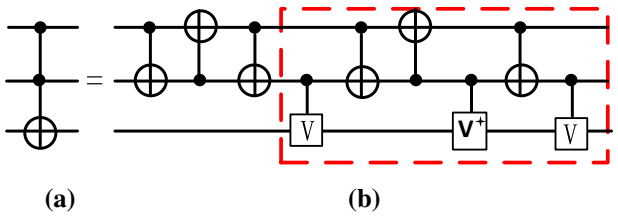


Fig. 4 Optimal LNN quantum circuit realization of NNT_3^1 gate. **a** NNT_3^1 gate and **b** its optimal LNN quantum circuit realization

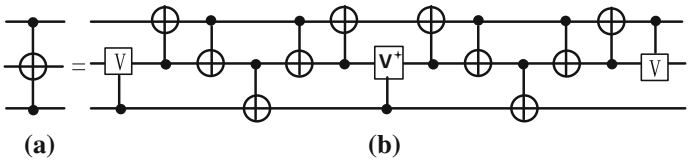


Fig. 5 Optimal LNN decomposition of NNT_3^2 gate. **a** NNT_3^2 gate and **b** Its optimal LNN decomposition

If the target bit is in the middle position of NNT_3, namely NNT_3^2, its LNN realization is as shown in Fig. 5, its quantum cost is 13, and the circuit might be entangled [20], so for this case, we convert NNT_3^2 into NNT_3^1 or NNT_3^3.

3 LNN arrangement of the MCT gate with $m \geq 3$

Definition 1 If an MCT gate is an LNN elementary quantum gate or it can be substituted directly by the cascade of LNN elementary quantum gates, then the arrangement of the input lines is defined as LNN arrangement of the MCT gate.

For an MCT circuit, n is the number of input lines, i.e., the size of the circuit. LNN arrangement of a NOT gate is itself, and for a CNOT gate, there are two forms: The position of the target bit is below or above, and LNN arrangement of the original Toffoli gate is NNT_3^1 or NNT_3^3. Suppose m is the number of control bits of an MCT gate; LNN arrangement of an MCT gates with $m \geq 3$ is discussed as below.

For the convenience of description, we denote the MCT gate with $n \geq 5$, $m \in \{3, \ldots, \frac{n}{2}\}$ as MCT_{m+1}^1 gate and the MCT gate with $n \geq 5$, $m > \frac{n}{2}$, and $m < n-1$ as MCT_{m+1}^2 gate.

As described in [16], an MCT_{m+1}^1 gate can be equal to a network consisting of $4(m-2)$ MCT_3 gates; Figure 6 is illustrated for $n=9$ and $m=5$.

Lemma 1 *An* MCT_{m+1}^1 *gate can be realized by a network consisting of* $4\,(m-2)$ NNT_3 *gates.*

Proof In order to satisfy the underlying LNN constraints, the order of input lines is rearranged, which can make all the MCT_3 gates in Fig. 6b become NNT_3 gates, as shown in Fig. 7 (illustrated for $n=9$ and $m=5$). □

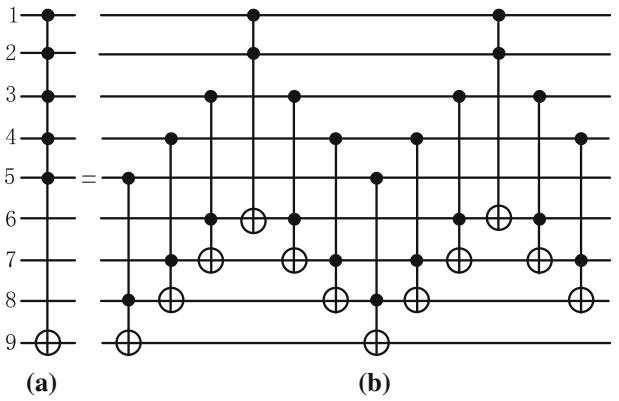


Fig. 6 MCT^1 gate cascaded by the original Toffoli gates. **a** MCT^1 gate and **b** its equivalent form cascaded by MCT_3 gates

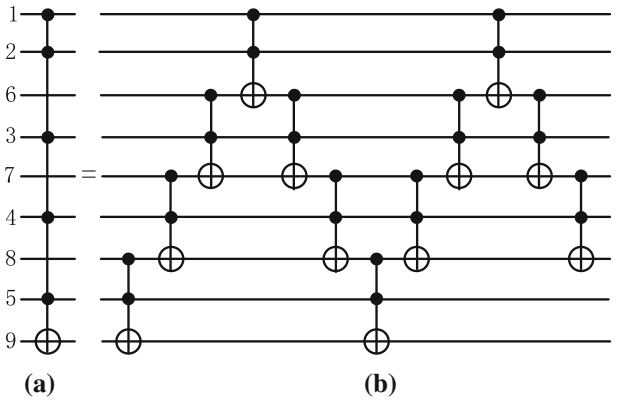


Fig. 7 MCT^1 gate cascaded by NNT_3 gates. **a** MCT^1 gate and **b** its equivalent form cascaded by NNT_3 gates

As stated in [16], for any $n \geq 5$, and $m_1 \in \{2, \ldots, n-3\}$, then an MCT_{n-1} gate can be decomposed into two MCT_{m_1+1} gates and two MCT_{n-m_1} gates. Specially, that $n=9$ and $m_1=5$ is illustrated in Fig. 8. We can find that there is an empty line and the sum of the number of control bits of MCT_{m_1+1} and MCT_{n-m_1} is given by

$$m_1 + n - m_1 - 1 = n - 1. \tag{3}$$

Lemma 2 *For any* $n \geq 6, m_1 \in \{2, \ldots, n-3\}$, *an* MCT_{n-1} *gate can be decomposed into two* MCT_{m_1+1} *gates and two* MCT_{m_2+1} *gates, if* $m_1 = \frac{n}{2}$, *then* $m_2 \geq 2$, *and* $m_2 < \frac{n}{2}$.

Proof For $n \in [2, 4]$, there is no need to decompose, if $n = 5$; it conforms to Lemma 1.

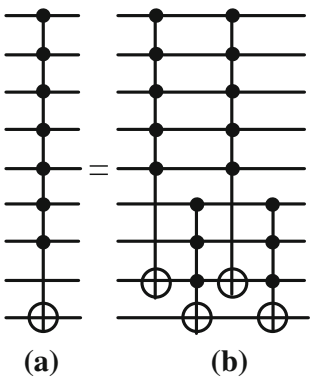


Fig. 8 MCT gate cascaded by smaller size MCT gates. **a** MCT gate and **b** its equivalent form with smaller size MCT gates

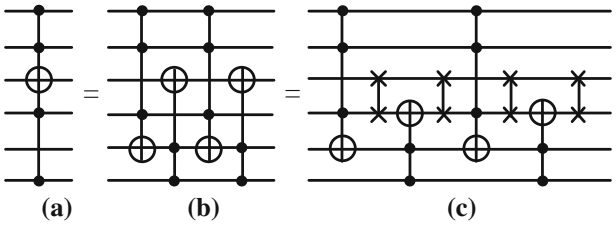


Fig. 9 NNT-based LNN realization of MCT^2 gate ($n=6$ and $m=4$). **a** MCT^2 gate, **b** cascaded by MCT^1 gates, and **c** its LNN realization

For $n \geq 6$, $m_1 = \frac{n}{2} \geq 3$. Because $m_1 + m_2 = n - 1$,

$$m_2 = n - \frac{n}{2} - 1 = \frac{n}{2} - 1 < \frac{n}{2}. \tag{4}$$

If $n=6$ or $n=7$, $m_2 = 2$, else if $n \geq 8$, $m_2 \geq 3$. □

Theorem 1 *For any $n \geq 8$, the LNN realization of a MCT^2_{n-1} gate consists of $8(n-5)$ NNT_3 gates and $4(n \mod 2+1)$ NNSwap gates, that is $72(n-5)+12(n \mod 2+1)$ elementary quantum gates if the NCV library is used. If $n \in \{6, 7\}$, two extra NNT_3 gates are needed.*

Proof (1) n is an even number

Figure 9 is illustrated for $n=6$ and $m=4$, and the number of NNT_3 gates (denoted as $\#NNT_3$) is given by

$$\begin{aligned} 2 \times [4(m_1 - 2)] + 2 &= 8(m_1 - 2) + 2 \\ &= 8(n - 5) + 2, \end{aligned} \tag{5}$$

and the number of NNSwap gates (#NNSwap) is 4.

Figure 10 is illustrated for $n=8$ and $m=6$, $\#NNT_3$ is

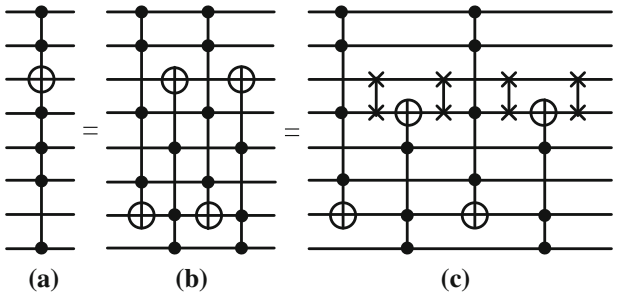


Fig. 10 NNT-based LNN realization of MCT^2 gate ($n=8$ and $m=6$). **a** MCT^2 gate, **b** cascaded by MCT^1 gates, and **c** its LNN realization

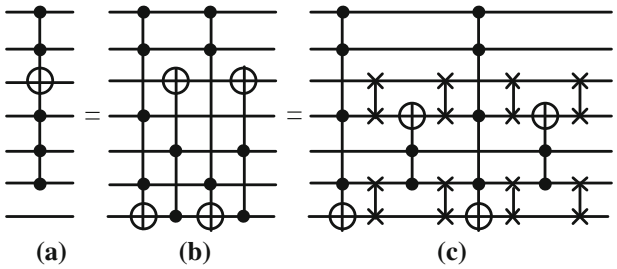


Fig. 11 NNT-based LNN realization of MCT^2 gate ($n=7$ and $=5$). **a** MCT^2 gate, **b** cascaded by MCT^1 gates, and **c** its LNN realization

$$2 \times [4(m_1 - 2) + 4(m_2 - 2)] = 8(m_1 + m_2) - 32 = 8(n - 5), \quad (6)$$

and #NNSwap is 4.

(2) n is an odd number

Figure 11 is illustrated for $n=7$ and $m=5$, $\#NNT_3$ is

$$2 \times [4(m_1 - 2)] + 2 = 8(m_1 - 2) + 2 = 8(n - 5) + 2, \quad (7)$$

and #NNSwap is 8.

Figure 12 is illustrated for $n=9$ and $m=7$, $\#NNT_3$ is

$$2 \times [4(m_1 - 2) + 4(m_2 - 2)] = 8(m_1 + m_2) - 32 = 8(n - 5), \quad (8)$$

and #NNSwap is 8.

In summary, $\#NNT_3$ is $8(n-5)$ except for $n=6$ or $n=7$; in these two cases, $\#NNT_3$ needs to add 2.

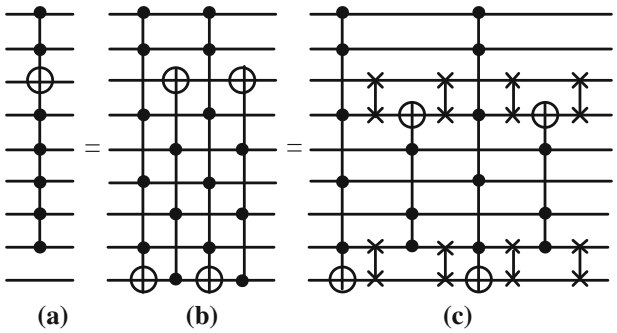


Fig. 12 NNT-based LNN realization of MCT^2 gate ($n=9$ and $m=7$). **a** MCT^2 gate, **b** cascaded by MCT^1 gates, and **c** its LNN realization

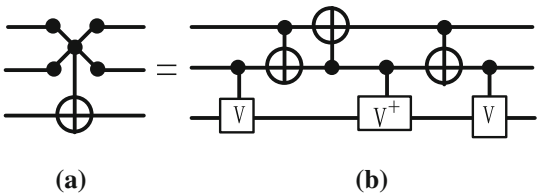


Fig. 13 NNTS gate and its optimal LNN quantum circuit. **a** graphical representation and **b** its equivalent LNN sequence

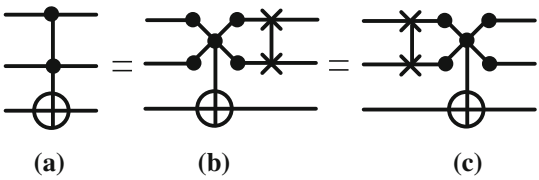


Fig. 14 Relation between the NNT_3 gate and the NNTS gate. **a** NNT_3 gate, **b** the first relation, and **c** the second relation

When n is an even number, #NNSwap is 4, and when n is an odd number, #NNSwap is 8, so #NNSwap is $4\,(n \bmod 2+1)$, where mod represents the remainder operator.

The QC of NNT_3 gate is 9, and that of NNSwap gate is 3, so the total QC of the LNN substitution is given by

$$72\,(n-5)+12\,(n \bmod 2+1)\;(n \geq 8) \tag{9}$$

and

$$72\,(n-5)+18+12\,(n \bmod 2+1)\;(n \in [6,7])\,. \tag{10}$$

□

Definition 2 The gate sequence shown in Fig. 13b is defined as NNTS gate.

It is a subsequence of Fig. 4b, and the relation between NNT_3 gate and NNTS gate is shown in Fig. 14.

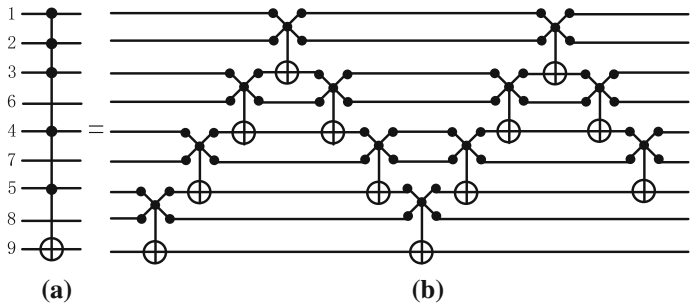


Fig. 15 MCT^1 gate cascaded by NNTS gates. **a** LNN arrangement of MCT^1 gate and **b** its equivalent form cascaded by NNTS gates

Definition 3 For an MCT gate g, the number of qubits from the first non-empty line to the last non-empty line is defined as real width, denoted as $RealW(g)$, and the least number of qubits for its LNN arrangement is termed as LNN width, denoted as $LNNW(g)$.

Such as the first CNOT gate in Fig. 1, its real width is 4, and its LNN width is 2.

Lemma 3 *For any* $n \geq 5$, *and* $m \in \left\{3, \ldots, \frac{n}{2}\right\}$, *an* MCT^1_{m+1} *gate is equivalent to a network consisting of* $4\,(m-2)$ *NNTS gates, QC is* $24\,(m-2)$, *and its LNN width is* $2m-1$.

Proof In Fig. 7b, each NNT_3 gate is substituted by an NNTS gate and an NNSwap gate; meanwhile, the order of the input lines is rearranged, and all the NNSwap gates can be omitted, as shown in Fig. 15b. □

Definition 4 For an MCT^1 gate, if it takes up consecutive $2m-1$ qubits denoted as $\{x_1, x_2, \ldots, x_{2m-1}\}$, if x_1 and x_2 are control bits, x_{2m-1} is target bit, and for $i \in \{3, 2m-2\}$, if i is an odd number and x_i is control bit, x_{i+1} is empty line, then the order or reverse order of the input lines is defined as LNN arrangement of the MCT^1 gate.

Theorem 2 *For any* $n \geq 6$, $m > \frac{n}{2}$, $m < n-1$, *the LNN realization of an* MCT^2_{n-1} *gate consists of* $8\,(m-3)$ NNTS *and* $8\,(m \bmod 2)$ *NNSwap gates, that is* $48\,(m-3)+24\,(n \bmod 2)$ *elementary quantum gates if the NCV library is used, its LNN width is* $m+2$. *As a special case, if* $m=4$ *or* $m=5$, *two extra* NNT_3 *gates are needed.*

Proof (1) n is an even number

Figure 16 is illustrated for n=6 and m=4, the number of NNTS gates (denoted as #NNTS) is

$$\begin{aligned} 2 \times [4 \times (m_1 - 2)] &= 8\,(m_1 - 2) \\ &= 8\,(n-5)\,, \end{aligned} \tag{11}$$

and $\#\mathrm{NNT}_3$ is 2, and quantum cost is 66.

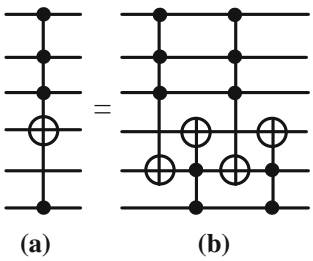


Fig. 16 NNTS-based LNN realization of MCT^2 gate ($n=6$ and $m=4$). **a** MCT^2 gate and **b** its LNN realization

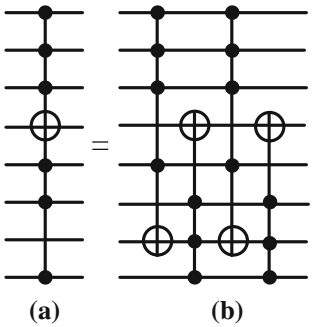


Fig. 17 NNTS-based LNN realization of MCT^2 gate ($n=8$ and $m=6$). **a** MCT^2 gate and **b** its LNN realization

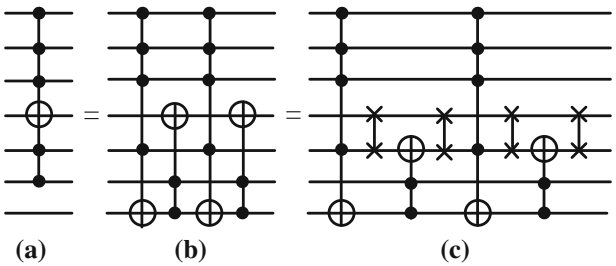


Fig. 18 NNTS-based LNN realization of MCT^2 gate ($n=7$ and $m=5$). **a** MCT^2 gate, **b** cascaded by MCT^1 gates, and **c** its LNN realization

Figure 17 is illustrated for $n=8$ and $m=6$, #NNTS is

$$\begin{aligned} 2 \times [4\,(m_1 - 2) + 4\,(m_2 - 2)] &= 8\,(m_1 + m_2) - 32 \\ &= 8\,(n - 5)\,, \end{aligned} \tag{12}$$

and QC is $48(n-5)$.

(2) n is an odd number

Figure 18 is illustrated for $n=7$ and $m=5$, #NNTS is

$$\begin{aligned} 2 \times [4 \times (m_1 - 2)] &= 8\,(m_1 - 2) \\ &= 8\,(n - 5)\,, \end{aligned} \tag{13}$$

$\#NNT_3$ is 2, #NNSwap is 4, and QC is 126.

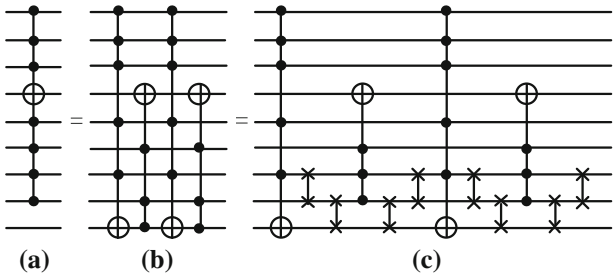


Fig. 19 NNTS-based LNN realization of MCT^2 gate ($n=9$ and $m=7$). **a** MCT^2 gate, **b** cascaded by MCT^1 gates, and **c** its LNN realization

Figure 19 is illustrated for $n=9$ and $m=7$, #NNTS is

$$\begin{aligned} 2 \times [4\,(m_1 - 2) + 4\,(m_2 - 2)] &= 8\,(m_1 + m_2) - 32 \\ &= 8\,(n - 5)\,, \end{aligned} \tag{14}$$

and #NNSwap is 8.
In summary, #NNTS is $8(n-5)$, and for $n=6$ or $n=7$, two NNT_3 gates are extra needed.
When n is an even number, NNSwap gate is not needed, and when n is an odd number, #NNSwap is 8 except for $n=7$; in this special case, only four NNSwap gates are needed.
The QC of NNTS is 6, and that of NNSwap is 3, so the total QC is given by

$$48\,(n - 5) + 24\,(n \bmod 2)\ (n \geq 8) \tag{15}$$

and

$$48\,(n - 5) + 18 + 12\,(n \bmod 2)\ (n \in [6, 7]) \tag{16}$$

□

Comparing Theorem 1 to Theorem 2, for the same MCT^2_{n-1} gate, the QC based on NNTS is less than that based on NNT_3.

Definition 5 For an MCT^2 gate, if it takes up consecutive $m+2$ qubits denoted as $\{x_1, x_2, \ldots, x_{m+2}\}$, if x_1, x_2, and x_3 are control bits, x_4 is target bit, and for $i \in [5, m]$, x_i are all control bits, if m is an even number, x_{m+1} is empty line, and x_{m+2} is control bit, otherwise, x_{m+1} is control bit and x_{m+2} is empty line, then the order or reverse order of the input lines is defined as LNN arrangement of the MCT^2 gate.

Definition 6 An MCT gate with LNN arrangement is called LNN MCT gate; otherwise, it is termed as non-LNN MCT gate.

4 Communication overhead calculation model in line rearrangement

In Sect. 3, if we want to get the LNN arrangement of an MCT gate, we need to rearrange the order of the lines, the number of NNSwaps gates in the transformation is called communication overhead, and it is calculated by the following proposed algorithm.

Definition 7 For an MCT(C; t) gate, a vector (b_1, b_2, …, b_n) is used to denote the gate, the value of b_i is defined as

$$b_i = \begin{cases} 1, b_i \in C \\ 2, b_i = t \\ 0, \text{other} \end{cases}, \tag{17}$$

and it is defined as the vector representation of an MCT gate.

Suppose the original arrangement is O_v=(o_1, o_2, …, o_n), the new arrangement is D_v=(d_1, d_2, …, d_n), and #NNSwap for each d_i from the original arrangement to the new arrangement is represented by S_v.

Definition 8 For any d_i ($1 \le i \le n$) in D_v, suppose its position in O_v is j, denoted as $\text{Loc}(O_v, d_i) = j$, the "move vector" M_v is defined, and the ith value of M_v is $j - i$, that is

$$M_v(i) = \text{Loc}(O_v, d_i) - i. \tag{18}$$

Definition 9 For any d_i ($1 \le i \le n$) in D_v, suppose $\text{Loc}(O_v, d_i) = j$, the "inversion vector" I_v is defined, and the ith value of I_v is the number of elements which are after the o_j in O_v but they are before d_i in D_v, i.e.,

$$I_v(i) = |\{o_k | \text{Loc}(O_v, o_k) > j \quad \text{and} \quad \text{Loc}(D_v, o_k) < i\}|, \tag{19}$$

symbol "|" represents the number of elements which satisfy the condition.

Theorem 3 *For any MCT gate with $m \ge 2$, the original arrangement is $O_v = (o_1, o_2, \ldots, o_n)$, the new arrangement is $D_v = (d_1, d_2, \ldots, d_n)$, then*

$$S_v(i) = M_v(i) + I_v(i), \tag{20}$$

and the total #NNSwap from O_v to D_v is defined as

$$\text{nSwaps} = \sum_{i=1}^{n} S_v(i). \tag{21}$$

Proof (1) Move forward, as shown in Fig. 20.

The elements before d_i are rearranged, and the location of the rest elements is i or after i in O_v. The element after o_j such as o_k moves toward d_s, which makes o_j move right, so if there are p gates after o_j moves before d_i, o_j moves right p

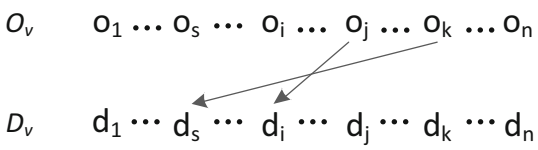


Fig. 20 O_j moves forward to d_i

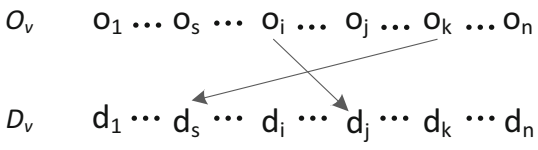


Fig. 21 o_i moves backward to d_j

times, and when o_j is moved to d_i, it needs $j - i + p$ NNSwap gates. Because $M_v(i) = j - i$, $I_v(i) = p$, so $S_v(i) = M_v(i) + I_v(i)$.

(2) Move backward, as shown in Fig. 21.
The elements before d_j are rearranged, and the location of the rest elements is j or after j in O_v. The element after o_i such as o_k moves toward d_s, which makes o_i moves right, so if there are p gates after o_i moves before d_j, o_i moves right p times, and when o_i is moved to d_j, it needs $p - (j - \mathrm{i}) = i - j + p$ NNSwap gates. $M_v(i) = i - j$ is a negative value, $I_v(i) = p$, so $S_v(i) = M_v(i) + I_v(i)$. Combining case 1 and case 2, we can draw a conclusion that $M_v(i) = \mathrm{Loc}(O_v, d_i) - i$, $S_v(i) = M_v(i) + I_v(i)$, and $\mathrm{nSwaps} = \sum_{i=1}^{n} S_v(i)$. □

Algorithm 1. Communication overhead calculation algorithm

Input:
The old arrangement of a non-LNN MCT gate $g(m \geq 2)$ O_v and the new arrangement D_v.

Output:
#NNSwap and all the specific NNSwap gates.

S1: Get M_v and I_v according to Definition 8 and Definintion 9, calculate $S_v(i) = M_v(i) + I_v(i)(1 \leq i \leq n)$;
S2: Set $i = 1$, gcount=0, initialize the stack Swaps;
S3: If $i > n$, go to **S**6, else set $k = S_v(i)$;
S4: If $k \neq 0$, push NNSwap gate $(i + k - 1) \leftrightarrow (i + k)$ to Swaps, gcount=gcount+1, $k = k - 1$, go to **S4**;
S5: $i = i + 1$, go to **S3**;
S6: Return gcount and Swaps.

Example 1 For an MCT gate (n=9, m=3), the serial number of input lines is denoted as 1, 2, …, n, its original arrangement is O_v=(1, 2, 0, 0, 0, 0, 1, 0, 1), and the new one is D_v=(0, 0, 0, 0, 2, 0, 1, 1, 1), and from the original to the new, the #NNSwap and specific SWAP gates are as follows.

The mapping relation is shown in Fig. 22. The "move vector" M_v=(2, 2, 2, 2, −3, 2, −6, −1, 0), and the "inversion vector" I_v=(0, 0, 0, 0, 4, 0, 6, 1, 0), so S_v=(2, 2, 2,

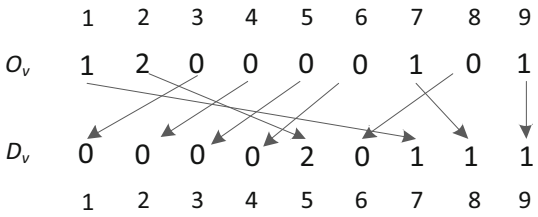


Fig. 22 Mapping from the original arrangement to the new arrangement

Table 1 Total number of LNN arrangement of each type of MCT gate

Gate type	LNN width	#Arrangement
CNOT($m=1$)	2	$2(n-1)$
Toffoli($m=2$)	3	$2(n-2)$
MCT^1 $\left(3 \le m \le \frac{n}{2}\right)$	$2m-1$	$2(n-2m+2)$
MCT^2 $\left(\frac{n}{2} < m < n-1\right)$	$m+2$	$2(n-m-1)$

2, 1, 2, 0, 0, 0); the total number of inserted NNSwap gates is 11. The inserted SWAP gates are $2 \leftrightarrow 3$, $1 \leftrightarrow 2$, $3 \leftrightarrow 4$, $2 \leftrightarrow 3$, $4 \leftrightarrow 5$, $3 \leftrightarrow 4$, $5 \leftrightarrow 6$, $4 \leftrightarrow 5$, $5 \leftrightarrow 6$, $7 \leftrightarrow 8$, $6 \leftrightarrow 7$.

5 LNN quantum circuit generation based on LNN arrangement of MCT gates

The general method of converting an MCT circuit into an LNN quantum circuit includes: (1) For any MCT gate ($m \ge 2$), it is decomposed into the cascade of elementary quantum gates at first. (2) Ensure each quantum gate acts on adjacent qubits by inserting NNSwap gates. (3) Optimize the LNN quantum circuit. We have known that the decomposition method has a great influence on final quantum cost and it greatly increases the length of quantum circuits. In this paper, an LNN realization method of an MCT circuit is proposed from a new perspective, there is no need to decompose the MCT gate into elementary quantum gates, it only needs to obtain the LNN arrangement of each MCT gate, and then each LNN arrangement is substituted by its optimal LNN quantum circuit realization.

In an n-qubit quantum circuit, for an MCT gate, if its LNN width is k, its LNN arrangement is to select k consecutive lines from total n lines, each LNN arrangement has two directions, and the total impossible arrangement is $2(n-k+1)$. For each type of MCT gate, its total number of arrangements (#arrangement) is listed in Table 1. Especially, for the MCT^2 gate with $m=n-1$, one auxiliary line must be added.

During mapping, the circuit is scanned from beginning to end, for each MCT gate, consider its LNN arrangement instead of direct decomposition, each LNN arrangement of MCT^1 or MCT^2 gate is substituted by a sequence of NNTS and NNT_3 gates, then NNTS and NNT_3 gates are replaced by its optimal LNN quantum circuits, and finally, the LNN quantum circuit is obtained.

As stated before, an MCT gate with LNN width k has multiple options to locate the consecutive k qubits, even though we can take all cases into consideration and select one with the minimum #NNSwap as the final result; however, the time complexity is

large, and it is only suitable for small-scale circuits; when we solve the large-scale problem, we must seek some efficient strategies to speed up the solution.

Lemma 4 *For MCT gates* MCT $(C_1; t_1)$ *and* MCT $(C_2; t_2)$*, if* $t_1 \notin C_2$ *and* $t_2 \notin C_1$*, which means the target of one gate is not the control of the other gate, then the two MCT gates can be exchanged, which is called "move rule."*

For a non-LNN MCT gate ($m \geq 2$), how to select the LNN arrangement among multiple options? A local optimal LNN arrangement algorithm is given.

Algorithm 2. Local optimal LNN arrangement algorithm

Input:

A non-LNN MCT gate $g(m \geq 2)$.

Output:

A local optimal LNN arrangement of g and all NNSwap gates inserted before g.

S1: Obtain the first and the last non-empty line of g, represent them minline and maxline respectively;

S2: If $RealW(g) \geq LNNW(g)$ then set startpos=minline, endpos=maxline, go to **S4**;

S3: If $RealW(g) < LNNW(g)$ then set startpos=min(minline, maxline-nnbit+1),endpos=max(maxline, minline+nnbit-1), if startpos<0 then set startpos=0, if endpos≥n, then set endpos=n-1, nnbit represents LNN width;

S4: Find local optimal LNN arrangement D_v of g according to Theorem 3 in interval [startpos, endpos] with the minimum number of NNSwap gates;

S5: Utilize Algorithm 1 to get all the inserted SWAP gates in terms of the original arrangement O_v and the new found arrangement D_v;

S6: Return the local optimal LNN arrangement and all the NNSwap gates.

When the MCT circuit is traversed, for each of the non-LNN MCT gates ($m \geq 2$), its next gate is examined and judged whether it is an LNN gate and satisfies the move rule; if it is, then we interchange their order; otherwise, we consider only local optimum and find the optimal LNN arrangement of the current gate by inserting the minimal number of NNSwap gates; the LNN arrangement-based mapping algorithm is proposed.

For a non-LNN CNOT gate, there are three move directions: downward, upward and to the middle position, they have the same #NNSwap; however, different movement strategy has some influence on final results, so we try three types of specific methods: move to the top, to the bottom, and to the middle. For each move method, the following LNN arrangement-based mapping algorithm is executed.

Algorithm 3. LNN arrangement based mapping algorithm

Input:

A reversible MCT circuit $G = g_1 g_2 \cdots g_l$.

Output:

An LNN quantum circuit.

S1: Examine the MCT circuit and decide whether to add an auxiliary line;

S2: Traverse the circuit, $i = 1$;

S3: if $i \leq l$, go to **S4**, else go to **S9**;

S4: If g_i is an LNN gate then $i = i + 1$, go to **S3**, else go to **S5**;

S5: Judge whether g_{i+1} is an LNN gate, if it is and it can be exchanged with g_i, then exchange g_i and g_{i+1}, $i = i + 1$, go to **S5**;

S6: If g_i is a CNOT gate, we adopt one of the three move strategies specified currently;

S7: If g_i is an MCT gate with $m \geq 2$, we adopt Algorithm 2 to get the local optimal LNN arrangent;

S8: Set $i = i + 1$, go to **S3**;

S9: Return the total #NNSwap and quantum cost.

Definition 10 After the current non-LNN MCT gate is dealt with in Algorithm 3, the changed vector representations of its subsequent gates are recorded in a column of a matrix, which is called the matrix representation of gate arrangement transformation.

Example 3 The LNN arrangement realization by means of Algorithm 3 for optimal MCT circuit of benchmark 4gt5_75.

We dealt with the gates in Fig. 1 one by one, and its matrix representation of gate arrangement transformation is shown in Table 2. The first CNOT gate was made into LNN form, and two NNSwap gates were inserted before it. Meanwhile, we found that the third gate in Column 1 is an LNN gate and it can be interchanged with the second gate, so the positions of two gates were interchanged. Next, the gate of "20100" in Column 2 is transformed into "02100" in Column 3. Even though the last MCT gate satisfies the LNN condition now, it cannot move forward, then "10201" is converted to "01120," and change "11210" to "11102" which guarantees the least number of NNSwap gates needed for the current MCT gate. The final LNN arrangement realization of 4gt5_75 is shown in Fig. 23 with quantum cost 60.

6 Simplification methods

The NNSwap gates are inserted according to Algorithm 3, and the LNN arrangement of each gate is obtained, and then some circuit optimization methods are taken into account.

First, each NNT gate is replaced by the NNTS and NNSWAP gates. There are two types of substitution as Fig. 14, and the detailed substitution flow is shown in Table 3.

Table 2 Matrix representation of gate arrangement transformation

1	2	3	4	5
00021				
20100	00012			
00012	20100	02100		
01201	01201	10201	01120	
11102	11102	11102	11210	11102

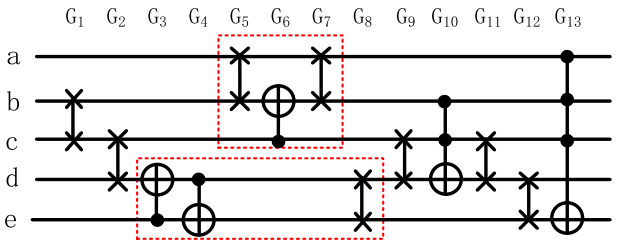


Fig. 23 LNN arrangement realization for 4gt5_75

Table 3 Substitution flow of the NNT gate

Order	Specific cases	Substitution
1	The first NNT gate	NNTS + NNSwap
2	There is an NNSwap gate at the right side of the NNT gate on its control bits	NNTS + NNSwap
3	There is an NNSwap gate at the left side of the NNT gate on its control bits	NNSwap + NNTS
4	There is an NNCNOT gate at the right side of the NNT gate on its control bits	NNTS + NNSwap
5	There is an NNCNOT gate at the left side of the NNT gate on its control bits	NNSwap + NNTS
6	The last NNT gate	NNSwap + NNTS

After substitution of the NNT gates, optimization rules for the transformed circuit are put forward.

Lemma 5 *Two identical NNCNOT gates can be deleted, and two identical NNSwap gates can be deleted.*

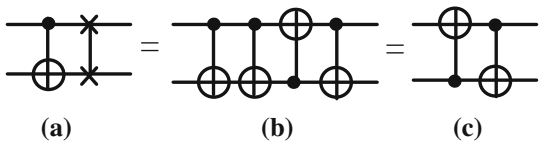


Fig. 24 Simplification of a CNOT and a SWAP with the same operated qubits. **a** The original layout, **b** the middle state, and **c** the simplified layout

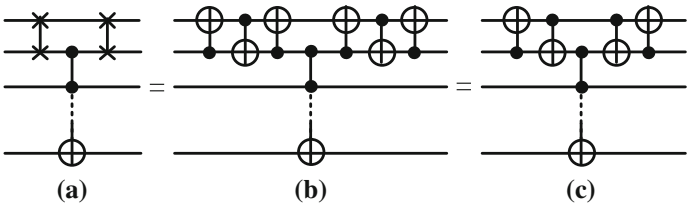


Fig. 25 Middle gate has only one control bit on one of q_i and q_j. **a** The original layout, **b** the middle state, and **c** the simplified layout

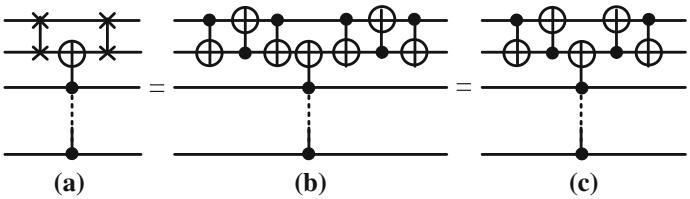


Fig. 26 Middle gate has only one target bit on one of q_i and q_j. **a** The original layout, **b** the middle state, and **c** the simplified layout

Proof Two identical CNOT gates can be deleted [3]. As shown in Fig. 2, the SWAP gate is cascaded by three CNOT gates, so two identical SWAP gates are equivalent to the identity circuit and can be omitted. □

Lemma 6 *An NNCNOT gate* $\mathrm{MCT}_2(\{q_i\}, q_j)$ *and an NNSwap gate act on the same two qubits; then, the sequence is equivalent to* $\mathrm{MCT}_2\left(\left\{q_j\right\}, q_i\right)$ *and* $\mathrm{MCT}_2(\{q_i\}, q_j)$.

Proof This case is shown in Fig. 24, an NNSwap gate is replaced by three NNCNOT gates and the leftmost two gates in Fig. 24b satisfy Lemma 5, so they can be deleted; Fig. 24c is obtained. □

Lemma 7 *Between two identical NNSwap gates which interact on* q_i *and* q_j*, the middle gate has only one control bit or one target bit on one of* q_i *and* q_j*, then two NNCNOT gates can be saved, and the quantum cost is reduced by 2.*

Proof The middle gate has a control bit on one of q_i and q_j as shown in Fig. 25, and a target bit on one of q_i and q_j as illustrated in Fig. 26. The NNSwap gate is replaced by three NNCNOT gates, and the middle two identical CNOT gates satisfy Lemma 4 which can be moved together and omitted. The simplified layout is shown in Figs. 25c and 26c, respectively.

The MCT circuit with LNN arrangement is simplified by means of substitution of NNT gates and application of above optimization rules. The detailed optimization procedure is described in Algorithm 4.

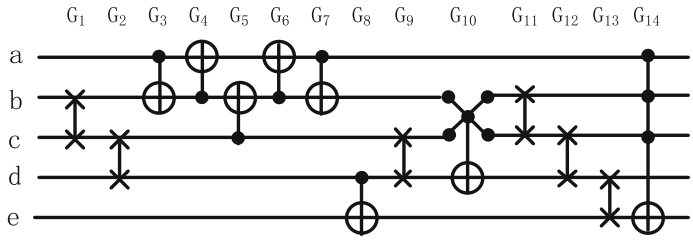


Fig. 27 Optimized circuit for 4gt5_75

Algorithm 4. Optimization algorithm

Input:

The MCT circuit $G = g_1 g_2 \cdots g_l$with LNN arrangement.

Output:

The optimized LNN quantum circuit.

S1: Replace each NNT gate according to TABLE III;

S2: Set flag=1;

S3: If flag=1 then traverse the circuit, set $i = 1$, flag=0;

S4: if $i > l$ or flag=0, go to **S8** ;

S5: If g_i is NNCNOT gate, if g_{i+1} is the same NNCNOT, then g_i and g_{i+1} are deleted, otherwise, if g_{i+1} is the NNSwap gate and in accord with the Lemma 6, then two NNCNOT gates can be saved, set flag=1, go to **S7**;

S6: If g_i is an NNSwap gate, if g_{i+1} is the same NNSwap gate, g_i and g_{i+1} are deleted, else if g_{i+1} is the NNCNOT gate and satisfy Lemma 6, then Lemma 6 is executed, else if g_{i+2} is the same NNSwap gate, and $g_i g_{i+1} g_{i+2}$ conforms to Lemma 7, then Lemma 7 is performed, set flag=1;

S7: $i = i + 1$, go to **S4**;

Example 4 Utilize the proposed optimization algorithm to simplify the LNN arrangement-based circuit in Fig. 23.

At first, we replace each NNT gate with NNTS and NNSwap gates. For the below red dotted frame in Fig. 23, G_4 and G_8 can be substituted by $MCT_2(\{e\}, d)$ and $MCT_2(\{d\}, e)$, G_3 and $MCT_2(\{e\}, d)$ satisfy Lemma 5; further, the sequence is simplified to $MCT_2(\{d\}, e)$, i.e., G_8 in Fig. 27. For the above red dotted frame in Fig. 23, the sequence conforms to Lemma 7 and they can be replaced by $G_3 \sim G_7$ in Fig. 27. Substitute G_{10}, G_{14}, and all SWAP gates with the optimal LNN quantum gate sequence, the final LNN quantum circuit is achieved with quantum cost 54, which is decreased by 6 compared to Fig. 23.

7 Experiments and result analysis

The experimental results are presented in this section to evaluate the efficiency and scalability of the proposed methods and compared with the LNN methods introduced in [14, 15]. The methods have been implemented in C++, and all the experiments have been conducted on the benchmarks available from [21] on a 2.5 GHZ dual-core i5 CPU with 4 GB RAM.

The existing methods for mapping decompose each reversible gate into a set of elementary quantum gates at first and then make each quantum gate satisfy LNN architecture by inserting NNSwap sequence, such as [14, 15]. In this paper, a novelty method was put forward, we obtained the LNN arrangement of each MCT gate firstly and converted each LNN arrangement into a sequence of LNN elementary quantum gates subsequently, and then some simplification strategies were given to reduce the quantum cost of the final circuit.

The number of inserted NNSwap gates and the final quantum cost were compared, and the specific results are shown in Tables 4 and 5, respectively.

We selected the same benchmark circuits as [14, 15] excluding decod24-v3_46, rd32-v0_67, and sys6-v0_144, which are not cascaded only by MCT gates. References [14, 15] performed LNN operations on the elementary quantum gates; however, we got the LNN realization of MCT circuits by means of LNN arrangement of each MCT gate. "#NNSwap" denotes the number of inserted NNSwap gates, and "#interswap" represents the number of inserted NNSwap gates in the case of odd m of MCT^2 gate in Theorem 2. "#totalswap" represents the number of total inserted NNSwap gates when we transform MCT circuit into its LNN arrangement form, including "#interswap." "Impr1_1" and "Impr1_2" show the results of "#totalswap" compared to "#NNSwap" in [14, 15], respectively.

The results in Table 4 demonstrate that the extra inserted NNSwap gates in our method are much less than [14, 15], excluding 4gt11_84. The average improvement rate compared to [14] is 68.23%, and to [15] is 49.65%. Because we only got the LNN arrangement-based MCT circuit by means of inserting NNSwap gates, instead of the final LNN quantum circuit, the decrease in #NNSwap is easy to see. However, it is sure that the less the #NNSwap, the less the final quantum cost.

In [14, 15], algorithms were operated on the elementary quantum gates directly, so their final quantum cost is that the original quantum cost of corresponding MCT gate adds the cost resulting from inserted NNSwap gates. In Table 5, the column of "distribution" represents the distribution of MCT gates ($m \geq 3$) in an MCT circuit; if there are no such gates in the circuit, then the number of MCT_3 gates is given. "ratio (%)" is the ratio of the MCT gates ($m \geq 3$) in the MCT circuit, QC_0 represents the quantum cost of LNN arrangement-based circuit, and QC_1 is the quantum cost of the final optimized circuit. "Impr2_1" and "Impr2_2" are the improvement rates that QC_0 and QC_1 compared to [15].

From the results listed in Table 5, even though a small fraction of results are a little worse than [15] on some small-scale MCT circuits, the vast majority of results manifest the effectiveness of our proposed algorithm, especially for circuits comprising MCT gates with high m value. The average improvement rate of QC_0 compared to [15] is 10.82%, and that of QC_1 compared to [15] is 16.95%. Meanwhile, the results show

Table 4 Comparison of the number of inserted NNSwap gates

Benchmark	n	gc	#NNSwap				Impr1. (%)	
			Reference [14]	Reference [15]	#Totalswap	#Interswap	Impr1_1	Impr1_2
3_17_13	3	6	6	4	2	0	66.67	50.00
4_49_17	4	12	20	12	9	0	55.00	25.00
4gt10-v1_81	5	6	30	20	8	0	73.33	60.00
4gt11_84	5	3	3	1	3	0	0.00	− 200.00
4gt12-v1_89	5	5	35	35	7	0	80.00	80.00
4gt13-v1_93	5	4	11	6	3	0	72.73	50.00
4gt4-v0_80	6	5	34	34	4	0	88.24	88.24
4gt5_75	5	5	17	12	8	0	52.94	33.33
4mod5-v1_23	5	8	16	9	7	0	56.25	22.22
4mod7-v0_95	5	6	28	21	6	0	78.57	71.43
aj-e11_165	5	6	39	36	9	0	76.92	75.00
alu-v4_36	5	7	23	18	6	0	73.91	66.67
ham7_104	7	23	84	68	29	0	65.48	57.35
hwb4_52	4	11	14	10	8	0	42.86	20.00
hwb5_55	5	24	79	63	27	0	65.82	57.14
hwb6_58	6	42	136	118	59	0	56.62	50.00
hwb7_62	8	331	3660	2128	721	56	80.30	66.12
hwb8_118	8	633	24,541	14,361	2172	48	91.15	84.88
hwb9_123	10	1959	36,837	21,166	5178	240	85.94	75.54
mod5adder_128	6	15	85	51	20	0	76.47	60.78
mod8-10_177	6	14	77	72	10	0	87.01	86.11
rd53_135	7	16	76	66	23	0	69.74	65.15
rd73_140	10	20	62	56	24	0	61.29	57.14
sym9_148	10	210	5480	3415	651	0	88.12	80.94
urf1_149	9	11,554	60,235	44,072	21,170	0	64.85	51.96
urf2_152	8	5030	25,502	17,670	8632	0	66.15	51.15
urf5_158	9	10,276	52,440	39,309	17,897	0	65.87	54.47

that our optimization algorithm can further reduce the final cost of LNN quantum circuit.

In order to reveal the relation between improvement rate and the maximal gate size of the circuit, a histogram is shown in Fig. 28. The abscissa denotes the maximal gate size in the MCT circuit, and the ordinate represents the average improvement rate of circuits with the same maximal gate size in Table 5. If the maximal gate size is 3, our LNN arrangement-based algorithm does not behave well as a whole; even after simplification, the results are still worse than [15]. However, if the maximal gate size and the ratio of MCT gates ($m>2$) are large, such as hwb8_118, the maximal gate size is 8 and the ratio of MCT gates ($m>2$) is 79.30%, the LNN arrangement-based method performs well, and the improvement rate reaches 42.76% compared to [15].

Table 5 Comparison of the final quantum cost

Benchmark	n	qc	gc	Distribution	Ratio (%) ($m>2$)	Quantum cost				Impr2. (%)	
						Reference [14]	Reference [15]	QC_0	QC_1	Impr2_1	Impr2_2
3_17_13	3	14	6	T_3:2	–	32	26	28	22	− 7.69	15.38
4_49_17	4	32	12	T_3:5	–	92	68	79	77	− 16.18	− 13.24
4gt10-v1_81	5	34	6	T_4:2	33.33	124	94	84	82	10.64	12.77
4gt11_84	5	7	3	T_3:1	–	16	10	20	20	− 100.00	− 100.00
4gt12-v1_89	5	42	5	T_4:1, T_5:1	40	147	147	114	110	22.45	25.17
4gt13-v1_93	5	17	4	T_4:1	25.00	50	35	36	36	− 2.86	− 2.86
4gt4-v0_80	6	44	5	T_5:1	20.00	146	146	90	90	38.36	38.36
4gt5_75	5	22	5	T_4:1	20.00	73	58	60	54	− 3.45	6.90
4mod5-v1_23	5	24	8	T_3:4	–	72	51	61	57	− 19.61	− 11.76
4mod7-v0_95	5	38	6	T_4:2	33.33	122	101	86	82	14.85	18.81
aj-e11_165	5	60	6	T_4:1	16.67	177	168	103	95	38.69	43.45
alu-v4_36	5	32	7	T_4:1	14.29	101	86	72	64	16.28	25.58
ham7_104	7	87	23	T_4:4	17.39	339	291	226	212	22.34	27.15
hwb4_52	4	23	11	T_3:3	–	65	53	59	49	− 11.32	7.55

Table 5 continued

Benchmark	n	qc	gc	Distribution	Ratio (%) ($m>2$)	Quantum cost				Impr2. (%)	
						Reference [14]	Reference [15]	QC_0	QC_1	Impr2_1	Impr2_2
hwb5_55	5	109	24	T_4:5	20.83	346	298	260	252	12.75	15.44
hwb6_58	6	146	42	T_4:4	9.52	554	500	415	363	17.00	27.40
hwb7_62	8	2663	331	T_4:40, T_5:34 T_6:14, T_7:2	27.19	13,643	9047	6866	6478	24.11	28.40
hwb8_118	8	14,260	633	T_4:201 T_5:166, T_6:98 T_7:31, T_8:6	79.30	87,883	57,343	32,822	32,232	42.76	43.79
hwb9_123	10	20,421	1959	T_4:77, T_5:121 T_6:122, T_7:74 T_8:30, T_9:6	21.95	130,932	83,919	51,807	49,267	38.27	41.29
mod5adder_128	6	87	15	T_4:4	26.67	342	240	207	199	13.75	17.08
mod8-10_177	6	109	14	T_4:1, T_5:2	21.43	340	325	221	213	32.00	34.46
rd53_135	7	78	16	T_4:1, T_5:1	12.50	306	276	203	195	26.45	29.35
rd73_140	10	76	20	T_3:14	–	262	244	204	144	16.39	40.98
sym9_148	10	4452	210	T_4:84, T_5:126	100	20,892	14,697	10,017	9849	31.84	32.99
urf1_149	9	57,770	11,554	T_3 (all)	–	238,475	189,986	167,496	156,540	11.84	17.60
urf2_152	8	25,150	5030	T_3 (all)	–	101,656	78,160	71,166	65,538	8.95	16.15
urf5_158	9	51,380	10,276	T_3 (all)	–	208,700	169,307	146,175	136,545	13.66	19.35

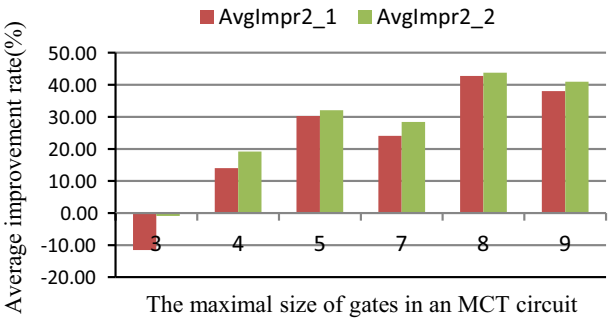


Fig. 28 Relation between the improvement rate and the maximal gate size

The results show that our proposed algorithm has good performance on circuits with large-size gates.

8 Conclusions

In order to map an MCT reversible circuit into an LNN quantum circuit, the NNT gates-based cascade form of the MCT gate with $m \geq 3$ was analyzed, and the new NNTS gate was introduced aiming at reducing the quantum cost, which can be used to construct the LNN cascade of the MCT gate. For each non-LNN gate in the MCT circuit, we get its LNN arrangement by inserting NNSwap sequence, and then the LNN arrangement-based mapping algorithm was proposed, and the results show that it can achieve pretty good results; moreover, some optimization methods were put forward, which can make the total quantum cost of the circuit decrease. The results of the put forward algorithms were compared to those of the existing methods based on direct decomposition, which shows that our method performs better on most of the benchmarks whether in #NNSwap or in quantum cost. In the future, we will consider the effect of current gate on its next several gates so as to reduce the #NNSwap during the mapping and give more effective simplification strategies to optimize the LNN quantum circuit.

Acknowledgements This work was supported by the Natural Science Foundation of Jiangsu Province under Grant BK20151274, General Project of Natural Science Research of Jiangsu Higher School under Grant 14KJB520033, and Postgraduate Research and Practice Innovation Program of Jiangsu Province under Grant KYCX17_1916.

References

1. Shor, P.: Algorithms for quantum computation: discrete logarithms and factoring. In: Proceedings of 35th Annual IEEE Symposium on FOCS, Santa Fe, NM, USA, pp. 124–134 (1994)
2. Grover, L.K.: A fast quantum mechanical algorithm for database search. In: Proceedings of 28th ACM Symposium on Theory of Computing, Philadelphia, PA, USA, pp. 212–219 (1996)
3. Nielsen, M.A., Chuang, I.L.: Quantum Computation and Quantum Information. Cambridge University Press, Cambridge (2000)

4. Lin, C., Sur-Kolay, S., Jha, N.K.: PAQCS: physical design-aware fault-tolerant quantum circuit synthesis. IEEE Trans. VLSI Syst. **23**(7), 1221–1234 (2015). https://doi.org/10.1109/TVLSI.2014.2337302
5. Shafaei, A., Saeedi, M., Pedram, M.: Determining the minimal number of swap gates for multidimensional nearest neighbor quantum circuits. In: Proceedings of ASP Design Automation Conference, Chiba, Japan, pp. 495–500 (2014)
6. Datta, K., Sengupta, I., Rahaman, H.: A post synthesis optimization technique for reversible circuits exploiting negative control lines. IEEE Trans. Comput. **64**(4), 1208–1214 (2015). https://doi.org/10.1109/TC.2014.2315641
7. Soeken, M., Wile, R., Hilken, C., Przigoda, N., Drechsler, R.: Synthesis of reversible circuits with minimal lines for large functions. In: Proceedings of ASP Design Automation Conference, Sydney, Australia, pp. 85–92 (2012)
8. Maslov, D., Dueck, G.W., Miller, D.M.: Techniques for the synthesis of reversible Toffoli networks. ACM Trans. Des. Autom. Electron. Syst. **12**(4), 42 (2007). https://doi.org/10.1145/1278349.1278355
9. Wille, R., Drechsler, R.: BDD-based synthesis of reversible logic for large functions. In: Proceedings of Design Automation Conference, San Francisco, CA, USA, pp. 270–275 (2009)
10. Kole, A., Datta, K.: Improved NCV gate realization of arbitrary size Toffoli gates. In: Proceedings of 2017 30th International Conference on VLSI Design and 2017 16th International Conference on Embedded Systems (VLSID), Hyderabad, India, pp. 289–294 (2017)
11. Miller, D., Wille, R., Sasanian, Z.: Elementary quantum gate realization for multiple-control Toffoli gates. In: Proceedings of International Symposium on Multiple-Valued Logic, Washington, DC, USA, pp. 288–293 (2011)
12. Chakrabatri, A., Sur-Kolay, S.: Nearest neighbour based synthesis of quantum Boolean circuits. Eng. Lett. **15**(2), 356–361 (2007)
13. Khan, M.H.A.: Cost reduction in nearest neighbour based synthesis of quantum Boolean circuits. Eng. Lett. **16**(1), 1–5 (2008)
14. Saeedi, M., Wille, R., Drechsler, R.: Synthesis of quantum circuits for linear nearest neighbor architectures. Quantum Inf. Process. **10**(3), 355–377 (2011). https://doi.org/10.1007/s11128-010-0201-2
15. Pedram, M., Shafaei, A.: Layout optimization for quantum circuits with linear nearest neighbor architectures. IEEE Circuits Syst. Mag. **16**(2), 62–74 (2016). https://doi.org/10.1109/MCAS.2016.2549950
16. Barenco, A., Bennet, C.H., Cleve, R., et al.: Elementary gates for quantum computation. Phys. Rev. A **52**, 3457–3467 (1995). https://doi.org/10.1103/PhysRevA.52.3457
17. Toffoli, T.: Reversible computing. Lecture Notes Comput. Sci. **85**(3), 632–644 (1980)
18. Fredkinand, E., Toffoli, T.: Conservative logic. Int. J. Theor. Phys. **21**(3–4), 219–253 (1982)
19. Peres, A.: Reversible logic and quantum computers. Phys. Rev. A **32**(6), 3266–3276 (1985)
20. Rahman, M., Dueck, G.: Synthesis of linear nearest neighbor quantum circuits. In: Proceedings of 10th International Workshop on Boolean Problems, Freiberg, Germany (2012)
21. Wille, R., Große, D., Teuber, L., et al.: RevLib: an online resource for reversible functions and reversible circuits. In: Proceedings of International Symposium on Multiple-Valued Logic, Washington, DC, USA, pp. 220–225 (2008). http://www.revlib.org